

XORT and its application in Chado and Apollo driven *Drosophila* genome annotation

Pinglei Zhou¹, Stan Letovsky^{1,2}, Christopher J Mungall³, David Emmert¹, Peili Zhang¹, Frank Smutniak¹, Scott Cain⁴, William Gelbart¹

¹FlyBase, The Biological laboratories, Harvard University, 16 Divinity Ave, Cambridge, MA 02138, USA

²Current address: , USA

³FlyBase-Berkeley, University of California, Berkeley, CA 94720-3200, USA

⁴Cold Spring Harbor Laboratory, Cold Spring Harbor, NY 11724, USA

Correspondence: Pinglei Zhou, Email: zhou@morgan.harvard.edu Fax: 617-496-1354

Abstract

We have developed XORT, an XML-database mapping system for data exchange. XORT's three major elements are an XML-database schema mapping specification, an XORT language for both data retrieval and data construction, and a collection of tools that are based on the specification to facilitate the mapping and data exchange. The mapping specification specifies the schema requirement, mapping, operation, and object reference mechanism; the XORT language is used to construct the dump specification; the tool collections comprise the data validation, data input, data output and data debug system. Because it originates from the Chado database, we denote the corresponding XML - Chado XML.

Our project has been using XORT for *Drosophila melanogaster* genome annotation in the context of a genome database, specifically, with a RDBMS schema named "Chado" and a GUI genome annotation tool named "Apollo". For that reason, we denote the corresponding XML Choad XML. Our results validate the architecture of XORT, and show that XORT provides excellent performance both in flexibility and extensibility. Despite its start as a system to manage an organism genome database, with its generic design, it can readily be extended to other database systems, as long the schema design satisfies minimal basic requirements. XORT can be accessed via GMOD at <http://www.gmod.org>

Motivation (or background)

The completion of the human genome sequence [1], along with the data available from numerous other organism database projects [2,3,4], provides a large challenge in data integration and management. Historically, model organism genomic data has been generated and maintained in different database systems for each organism (FlyBase, 2002; Blake, 2003; Moszer, 2002)[5,6,7]. The heterogeneity of these systems and their data representations has hindered the effective use of the available data (Baxeavanis, 2001)[8]. In response to the feedback from end users and the bioinformatics community, a NIH/USDA ARS-funded Generic Model Organism Database (GMOD, <http://www.gmod.org>) project was initiated, with its main goal to increase the homogeneity among different organism databases. Chado, a database schema produced by FlyBase, is a powerful ontology-driven and module organized structure (Mungal C, et al, 2006)[9], which has quickly gained popularity and has been adopted by several model organism databases (FlyBase,2006;] Chisholm RL et al, 2006; P Zhou, 2005) [10,11,12]. With extensive use of controlled vocabularies (primarily sequence ontology and gene ontology), Chado can easily store data reflecting the complicated hierarchical relationships common in biological systems, for example, the three-level hierarchy of the central gene model. Chado's modular structure increases its extensibility.

From the beginning of genomic sequencing, genome browsers and annotation editors have been crucial tools for visualizing and modifying the data produced by gene annotation. Several tools have been developed to facilitate the sequence visualization and annotation in different organisms. These include stand-alone applications such as Genotator (N.L. Harris, 1997)[13] , Artemis (K. Rutherford, 2000) [14], RiceGASS (K. Sakata, 2000) [15], Apollo () [16], SynBrowser (Pan , 2005) [17], and web-based systems, such as GESTALTA (G. Glusman, 2000) [18], GBrowse [19], Human Genome Browser (W. James Kent, 2002) [20], and Microbial Genome Viewer (Robert Kerkhoven, 2004) [21]. Apollo and GBrowse have been the primary tools used by FlyBase for Drosophila genome annotation and visualization even before the appearance of Chado. Now with a mature Chado database, the importance of a systematic and standardized middle layer between the database and end user applications becomes a priority.

To maximize the benefit of the powerful Chado schema, a middle layer is needed as a gateway between the database and applications. Minimum requirements for the middle layer should include a data storage

format that complements the flexibility and functionality of the database, has reasonable extensibility so that it can be easily incorporated into other applications with minimum transformation, and reasonable tools to facilitate the data input to and output from the database. Existing data formats used for biological information include GenBank, GFF3(L Stein et al) [22], and GAME XML (Mungall C, 2001)[23], however, these are limited to a subset of biological data. For example, GAME XML and GFF3 are for genomic features and cannot fully preserve the complicated hierarchical structure observed within phenotypic data. Additionally and importantly, these formats are static and cannot be used for incremental update for data objects.

This paper describes a system, XML Object Relational Transformation (XORT) that attempts to address these problems by (1) Defining an XML data format specification as “gateway” between the database and application, (2). Creating a system framework to facilitate the transformation between database and application, (3) Building an XML specification collection for typical use cases likely to be encountered in any organism database project and (4) Application of the system to annotation of the *Drosophila melanogaster* genome as a case study.

Result

Initially all the *Drosophila melanogaster* genome annotations were stored in a relational database called “Genome annotation database of Fly” (Gadfly) (Mungall C, 2002) [23]. FlyBase has recently migrated these annotations into a Chado database and built a new annotation system. This process made extensive use of XORT and therefore provides a robust evaluation of the architecture and performance of XORT.

Case study: An annotation system for *Drosophila melanogaster* using XORT, Apollo and Chado.

Chado XML representation

A central problem which any biological database schema cannot avoid is how to represent the central dogma gene model. In Chado, all objects of the gene model are stored as feature tuples. The hierarchical structures within these objects are stored as feature_relationship tuples. According to the mapping rule,

there are many ways to represent all of those relationships in Chado XML. However, the way which can explicitly demonstrate the three-tiered hierarchical structure is preferred as shown in Figure 1.

XORT language and dump specification

In this paper, we present the syntax and semantics of the XORT language. We introduce the XML document (dump specification) that is used to guide the XORT dumper to exports data from the database into customized XML format. The XORT dumping mechanism has the capability to generate XML documents that preserve the hierarchical database object relationships. Applying such a standard facilitates software development and maintenance by supporting data exchange among database/XML documents. The implementation of the Chado sequence module has been broadly adopted and can be used as a model to demonstrate the application of XORT language. The XML dump specification for the central gene model is shown in Figure 2. The corresponding operations on the database are shown in Figure 3.

Template repository

While all Chado users are urged to adopt a standard implementation endorsed by GMOD, we recognize the need for flexibility and heterogeneity and therefore provide the function to export the same data from different implementations. For this purpose, we have developed many templates that reside in our dump specification repository, including many that can be of common use, and some that are FlyBase project specific. Specific examples include; 1. templates for Apollo, 2. templates for JavaSean, 3. templates for comparative genome analysis, 4. templates for common gene page development. Chado XML generated from these templates can be converted into other standard formats via specific converters, or via other XML tools, e.g. XSLT.

Schematic representation of the annotation framework

Figure 4 diagrammatically depicts the overall architecture of our annotation framework. It consists of a few loosely coupled packages: Apollo, a genome annotation tool, which was developed by Ensembl and the BDGP, Chado, a backbone relational database, the XORT-transformation system between Apollo and the

database and the XORT template repository, which contains all wrappers for different projects. The stepwise annotation and data management procedure is described below.

Producing mini-Chado XML from Chado for Apollo annotation. The annotation process starts with annotators using a tool to produce, from the Chado database, a Chado XML file consisting of a defined region of the genome and the overlapping set of features. The tool allows specification of the region of the genome to retrieve and if aligned computational analyses should also be retrieved along with the annotated genomic features. Based on the input parameters, the dumper uses the corresponding dumper specification (Apollo dump spec) to translate the request into SQL and retrieve data from Chado, yielding output in Chado XML that conforms to specified constraints. The data is loaded into Apollo and edited by the curator. The results are then saved out of Apollo into Chado XML.

Loading edited Chado XML into Chado. The XORT Loader applies operational Chado XML (with default operation “force”). During this process, database triggers will be fired to handle the necessary changes inside the database. For example, triggers will assign some FlyBase specific identifiers to *de novo* annotated features. Triggers will also clean the orphan transcript(s), exon(s) and protein(s) if a gene feature is deleted.

Generating reporting XML Different dump specifications allow the updated data to be exported in XML having different structures. For example, different information is included in data for the Apollo editor, for JavaSean and for FlyBase web reports.

XORT Implementation

The XORT Package is written in perl. It has been tested on Sun Solaris systems, and the Chado database is implemented in Postgres. XORT contains a validator, a loader, a dumper and a diff utility. The validator verifies correct file format and identifies common syntax and semantic errors that may occur in the XORT output XML file against a particular database schema. The XORT loader loads verified XORT output XML into the database and the XORT dumper exports data from the database into XORT output

XML guided by the customizable dumper specification. The XORT diff can determine the difference between two XORT output XML files in a knowledgeable way.

Methods and specification

Mapping specification rule

The complicated hierarchical structure in genomic data is a good fit to the hierarchical structure of XML documents. In the mapping between XML elements and database table/columns, nesting in XML documents represents the table/column, primary/foreign key relationships. In addition to the advantage of preserving relational information in XML, the database schema has a unique DTD and the XML content is forced to conform to this DTD, therefore, the Chado XML is indirectly conforming to the Chado database schema.

The definition of a relational schema consists of relations, attributes, primary/foreign/unique keys of the relations and the types of attributes. As a self-describing structure, XML defines the semantic tags, attribute, and hierarchical structure using a nested document structure. The mapping rules we present here include:

- a. All schema relations and attributes, except the internal serial number that is the primary key for each relationship, will be transformed into XML element tags. For the sake of clarity, we denote the element derived from a relation as a table element, whereas those from schema attributes are denoted column elements.
- b. The foreign key relationship will be expressed as nested sub-element in XML
- c. Since all Chado relations use internal serial numbers as the primary keys which are not included in the XML, each relation must have semantically meaningful unique keys to constrain each tuple.
- d. In addition, we define two XML attributes to facilitate our transformation and operational behavior, “id” and “op”. The “id” attribute is used to specify the uniqueness of that table element (or a tuple from the database), and can be referred to as a scalar value of a foreign key later in the same file once defined. The XML attribute “op” is used to specify an operation behavior of the XML document. There are five options for this attribute: select, delete, insert, update and force.

Algorithms to enforce the mapping rules

The above rules are enforced by the XORT validator algorithm, as follows.

For all children element of root element

 If table element (1)

 Save attribute value of id and op if any

 for all children of table element

 If column element and not foreign key, then save the value

 else if column element and foreign key

 If child is scalar value, verify that it's pre-defined and retrieve the id of this object

 else if child is table element, goto (1)

 else if table element (2)

 Verify if all previous sibling elements will satisfy the requirement of the parent table

 element, which includes all column elements which are part of the unique key constraint,

 and non-default non-null values. Finish the transaction for parent table, then goto (1).

 else if scalar value

 error

 else if view/function/pseudo table element, do nothing

 else, error

XMLized language –XML Object relational Translation (XORT) language

The syntax for XORT language is very similar to Chado schema driven XML. In addition to DTD validated Chado XML, it has two additional attributes to control the generator behavior. First, the attribute “test” controls when expansion occurs, and applies to column and table elements. The element “or” can be nested inside any column element that has a test attribute to create sets of acceptable values. Inside a column element, the attribute “test=yes” means that the applicability of the next outer table element is conditional on a successful match of this field. The test column element may contain several options based on the element data type. A primitive data type column can test a primitive data value and a foreign key column can test an object reference (with global or local recognition). In addition, more than one of the

above, each wrapped in an <or>..</or> element can be used to indicate a list of acceptable values. An empty test element indicates null and “test=no” is used to indicate negation of the test.

Secondly, attribute “dump” controls what to expand and only applies to table elements. It has four options: all, cols, select and ref. “all” means all columns for the corresponding table plus all linkage tables are included in the output, with references to all foreign keys in columns or linking tables. This is the default dumper for a root object. This default behavior can be overwritten by nested elements. “ref” generates a reference for an object, which means only the unique key(s) column of this tuple will be returned. “cols” specify all columns of the this table element. “select” will enumerate all columns which are explicitly specified. To specify a column to be retrieved, it is stated simply as an empty element. Linking tables can also be specified as empty elements, in which case the default behavior occurs i.e. column data plus foreign key references.

Using the feature table from the Chado database as an example to show the XORT language behavior:

This specification:

```
<feature dump="all">
  <uniquename test="yes"><or>v1.1</or><or>v1.2</or></uniquename>
  <seqlen test="no"><or>v2.1</or><or>v2.2</or></col2>
</feature>
```

is the SQL equivalent of :

```
select * from feature where (uniquename=v1.1 or uniquename=v1.2) AND NOT (seqlen=v2.1 or
seqlen=v2.2 ), and all other tuples from relations that have foreign keys referring to this feature record
nested within this feature, but after all column elements of feature.
```

Algorithms to apply XORT language to Chado database

If there is dump specification to control the dumper behavior

for all child element of root element of dump specification

save the value of attribute ‘dump’. Parse the query statement which either specified by element

‘_sql’ or column element with attribute ‘test’.

Retrieve all data for this table from database (1)

else

retrieve all data for that table(s) specified in command and format into Chado XML.

Discussion

In this paper, we describe XORT and its application in Chado-Apollo driven genome annotation. Our approach is novel in the following respects.

Its XML central data exchange format makes it easy to enforce standards within the large genome research community. XML provides a flexible syntax for representing semi-structured data and can be used as the medium to represent information extracted from all types of different sources, such as relational databases and textual information repositories (Adelberg B, 1998; Sahuguet et al. 1999)[24, 25].

Furthermore, XML documents are self-contained documents. They include their underlying domain model in the tag surrounding their data, so can be exchanged among different projects, as long as these projects adopt the same semantics underlying the domain model (Zhang H and Stroulia E. 2002)[26]. Our mapping design benefits from previous research (Andreas Behm, Alin Deutsch, E. Bertino)[27], but differs from those in the following aspects. Our proposal is, to the best of our knowledge, the first to combine the static transformation with the operation for data updates among different sites. Also because of its operation feature, it becomes a bi-directional communication system between the database and end users, unlike previous uni-directional retrieval systems (Bui, 2002) [28].

Typically, in the biological community, central dogma refers to the core representation of genes, transcripts, proteins, and the relationships between them. Because of the usual three or more levels of hierarchy, it can be difficult to retrieve this type of information using standard SQL. For example, to get all information for a gene, along with its transcripts, proteins and exons is a complex query requiring many

table joins. In addition, having XML output that can preserve the hierarchical structure allows it to be used by an XML server engine (XML server reference here) [29], or fed into an XML transformation tool, allowing further analysis. The XORT language we describe in this paper solves this problem by combining the query and structuring the output in XML format. The XORT language differs from others (Alin Deutsch, A query language for XML, S Abiteboul, 1997; P. Byneman, 1996; M Fernandez) [30,31,32,33] in several important respects. Specifically, XORT embeds queries in Chado XML format and because Chado XML indirectly conforms to Chado schema, and is self-descriptive, it minimizes data ambiguity. Additionally XORT is not just a query language, but also serves as a template to control the structure of output. Particularly, with its hierarchical control and its flexibility, it is perfectly suitable for complicated data, e.g. biological data. Finally, its schema-awareness and machine-generated primary key feature determines its recursion properties. More specifically, because the machine generated primary key is never exposed outside of the database environment, any object needs to be represented by its unique key(s). If any of its unique keys is also a foreign key, then the object will need to be recursively represented by the combinations of unique keys which it refers to.

Our annotation system separates the editing tools from the database. Others prefer to directly communicate between editor and database to minimize asynchrony. However, this intermediate layer, avoids possible mutational black box operations from editing tools and allows the use of additional quality control tools. This is predicted to significantly increase the database integrity and safety, and we argue that this out-weighs any disadvantages.

Though XORT was originally designed for the biological data specific relational schema Chado, it can be easily extended to any other relational database, assume the schema satisfies two very general conventions. First, all primary keys must be internal, auto-generated, numeric identifiers that are used to create foreign key relationships between tables. The value of this identifier is not guaranteed to be consistent across different database instances, even when the datasets are the same. For this reason, the value is never exposed to the end-user of the database and will be meaningless out of the database

environment. Second, each table must have a unique key, providing a way to represent a unique record outside of the database environment.

The Chado XML has been criticized for being too loosely constrained and thereby leading to XML that is difficult to validate semantically. This is a perceived weakness of the current DTD validation of Chado XML. A solution that is currently being perused is an algorithm to auto-generate a detailed XML schema based on the dumper specification that will allow a more meaningful referencing of the content of the XML documents. We also recognize that some of the exhaustive searches required by the dumper will slow down performance, and are investigating the use of ontology or controlled vocabularies to build an ontology knowledge base that is predicted to significantly speed up queries. A details discussion of this effort is outside the scope of this paper and will be reported elsewhere.

References

- [1] J.C. Venter, M.D. Adams, E.W. Myers, et al: The sequence of human genome. Science 291:1304-1351 2001
- [2] Waterston RH, Lindblad-Toh K, Birney E, et al: Initial sequencing and comparative analysis of the mouse genome. Nature. 420(6915):520-62. Dec 2002.
- [3] Adams MD, Celniker SE, Holt RA, et al: The genome sequence of *Drosophila melanogaster*. Science. 2000 Mar 24;287(5461):2185-95.
- [4] Gibbs RA, Weinstock GM, Metzker ML, et al: Genome sequence of the Brown Norway rat yields insights into mammalian evolution. Nature. 2004 Apr 1;428(6982):493-521.
- [17] Pan, X., Stein, L. and Brendel, V. 2005. SynBrowse: a Synteny Browser for Comparative Sequence Analysis. Bioinformatics 21: 3461-3468

[22] Stein LD, Mungall C, Shu S, Caudy M, Mangone M, Day A, Nickerson E, Stajich JE, Harris TW, Arva A, Lewis S. The generic genome browser: a building block for a model organism system database. *Genome Res.* 2002 12(10):1599-610.

[9] Christopher J Mungall, David Emmert, Pinglei Zhou et, al: Chado: a modular relational database schema for model organisms. *Genome Research.* to be published.

[25] Sahuguet and F. Azavant. Looking at the Web through XML Glasses. *Conference on Cooperative Information System.* 1999. 148-159.

[24] B. Adelberg. NoDoSE: A tool for semi-automatically extracting semi-structured data from text documents. In *Proc. Intl. Conference on Management of data.* 1998. 283-294.

Kip Canfield, James Scoraces, Mapping XML Documents into databases: A Data-Driven Framework for Bioinformatics Data Interchange.

[8] Baxevanis, A.D. 2001. Information retrieval from biological databases. *Methods biochem. Anal.* 43, 155-185.

[28] Alex A.T Bui, Gregory S Weinger, Susan J Barretta, John David N, Dionission, and Hooshang Kangarloo. An XML Gateway to partient data for medical research application.

[30] Alin Deutsch, Mary Fernandez, Daniela Florescu, Alon Levy, Dan Suciu, A query language for XML

[31] S. Abiteboul, D. Quass, J. McHugh, J. Widom, J. Wiener, 1997. The Lorel Query Language for Semistructured Data. *Internation Journal on Digital Libraries*, vol. 1, No. 1, pp.68-88.

- [32] P. Buneman, S. Davidson, G.Hillebrand, D.Suciu 1996. A query language and optimization techniques for unstructured data, Proceedings of ACM-SIGMOD International Conference on management of Data.
- [33] M. Fernandez, D. Florescu, A. Levy, D. Suciu. 1997. A Query Language for a Web-Site Management System, SIGMOD Record, vol. 26. No.3,pp. 4-11.
- [13] N.L. Harris. Genotator: a workbench for sequence annotation, Genome Research 1997. 7 (7) 754-765
- [15] K. Sakata. Y. Nagamura, H. Numa, B. Antonio, H. Nagasaki, A. Itonuma, W. Watanabe, Y. Shimizu, I. Horiuchi, T. Matsumoto, T. Sasaki, K. Higo, 1998. RiceGASS: an automated annotation system and database for rice genome sequence, Nucleic Acids Research 30(1) 98-102.
- [18] G. Glusman, D. Lancet, 2000. GESTALT: a workbench for automatic integration and visualization of large-scale genomic sequence analyses, bioinformatics 16(5): 482-483.
- [14] K. Rutherford, J. Parkhill, J. Crook, T. Horsnell, P. Rice, M.A. Rajandream, B. Barrell. 2000. ArtemisL sequence visualization and annotation. Bioinformatics 10(16) :944-945
- [21] Robert Kerkhoven, Frank H.T van Enckevort, Jos Boekhorst, Douwe Molenaar, Roland J. Siezen. 2004. Visualization for genomics: the Microbial Genome Viewer. Bioinformatics
- [20] W. James Kent, Charles W. Sugnet, Terrence S. Furey, Krishna M. Roskin, Tom H. Pringle, Alan M. Zahler, and David Haussler. 2002. The Human Genome Browser at UCSC. Genome Research, 996-1006.
- [12] Zhou, P., Emmert, D. and Zhang, P. 2005. Using Chado to Store Genome Annotation Data in Current Protocols in Bioinformatics (A. D. Baxevanis, D. B. Davison, R. D. M. Page, G. A. Petsko, L. D. Stein and G. D. Stormo, eds) pp 6.9.1 – 6.1.27. John Wiley & Sons Inc, Hoboken, NJ.

[11] Chisholm RL, Gaudet P, Just EM, Pilcher KE, Fey P, Merchant SN, Kibbe WA. dictyBase, the model organism database for *Dictyostelium discoideum*.

Nucleic Acids Res. 2006 Jan 1;34(Database issue):D423-7.

[10] Grumbling G, Strelets V, FlyBase. FlyBase: anatomical data, images and queries. Nucleic Acids Res. 2006 Jan 1;34(Database issue):D484-8.

[5] Blake JA, Richardson JE, Bult CJ, Kadin JA, Eppig JT; Mouse Genome Database Group. MGD: the Mouse Genome Database. Nucleic Acids Res. 2003 Jan 1;31(1):193-5.

[6] Flybase Consortium. 2002. The FlyBase database of the *Drosophila* genome projects and community literature. Nucleic Acid Res. 30(1):106-108.

[7] Ivan Moszer, Louis M Jones, Sandrine Moreira, Cecilia Fabry and Anoine Danchin. 2002. SubtiList: the reference database for the *Bacillus subtilis* genome. Nucleic Acids Res. 30(1):62-65

Lewis SE, Searle SM, Harris N, Gibson M, Lyer V, Richter J, Wiel C, Bayraktaroglu L, Birney E, Crosby MA, Kaminker JS, Matthews BB, Prochnik SE, Smithy CD, Tupy JL, Rubin GM, Misra S, Mungall CJ, Clamp ME. Apollo: a sequence annotation editor. Genome Biol. 2002;3(12):RESEARCH0082. Epub 2002 Dec 23.

Mungall CJ, Misra S, Berman BP, Carlson J, Frise E, Harris N, Marshall B, Shu S, Kaminker JS, Prochnik SE, Smith CD, Smith E, Tupy JL, Wiel C, Rubin GM, Lewis SE. An integrated computational pipeline and database to support whole-genome sequence annotation. Genome Biol. 2002;3(12):RESEARCH0081. Epub 2002 Dec 23. Review.

[26] Huaxin Zhang, Eleni Stroulia: Babel: An XML-Based Application Integration Framework. CAiSE

[27] Andreas Behm, Andreas Ge

Acknowledgements

This work is supported by the grant from National Institute of health (National Human Genome Research Institute) to FlyBase. We would like to thank Susan Russo for database administration, Andy Schroeder for providing many clues for case study from biologist point's view. Don Gelbart for giving many useful suggestions. Nomi Harris and Mark Gibson implemented the Chado XML adaptor for Apollo.